

Approve Once: A Governed Logic Database for Agentic Decision Support

ACM Summer School on Responsible AI
Group 3: Human-Centred, Agentic, Decision-Support AI

Abstract

Requirements for a “human in the loop” over consequential automated decisions are widespread, but per-decision oversight scales linearly with volume and degrades into rubber-stamping precisely where it matters most: where the reviewer cannot independently verify the output under review. We describe and implement an alternative, resting on two contributions that depend on each other. The first is an inspectable *logic database* that carries the decision logic: a terminating Datalog evaluation in which evidence is three-valued and uncertainty survives negation, missing facts yield exact probability bounds computed in two evaluations rather than 2^k , the interval width is turned into the single question worth asking, and every answer arrives as a proof that distinguishes what was established from what was assumed. The second is a governance layer over it: an agent may *propose* rules and facts but never infers, and human approval attaches to *rules* rather than to decisions. The order matters. Approving a rule is only meaningful if approval binds what the rule will do on every future case, which is a property of the engine, not of the workflow: determinism and a readable derivation are what make rule-level oversight possible at all. Because a rule governs every future case its body matches, oversight amortises: in a run over 1,000 real credit applications, sixteen acts of review governed 589 decisions. We report three further results. On a synthetic domain generated from a withheld rule set, the rule miner recovers all three generating rules exactly and ranks them first, alongside five plausible near-misses—giving, unusually, ground truth for *rules* rather than predictions. Encoding a written lending policy revealed that the policy was silent on 11% of cases, a gap its authors had not noticed and which the system surfaced only because it refuses to guess. And a governance invariant—that no critical decision may rest on an unapproved rule—is enforced structurally and audited across every decision, with zero violations. We are explicit about what we have *not* shown: no human has yet reviewed a rule in this system, and we describe the experiment that would settle it.

Keywords

meaningful human control, human oversight, neurosymbolic systems, Datalog, explainability, agentic AI, EU AI Act Article 14

1 The problem with a human in the loop

Regulatory and organisational responses to automated decision-making converge on a common remedy: place a human in the loop. EU AI Act Article 14 requires that high-risk systems be subject to effective human oversight. The difficulty is that the remedy is usually operationalised *per decision*—a reviewer approves or rejects individual outputs—and per-decision oversight has two structural problems.

First, it does not scale. Oversight cost is flat at one review per decision, however large the caseload grows.

Second, and more seriously, it degrades into ceremony exactly where it is most needed. Where the reviewer cannot independently verify the output—because the model’s reasoning is unavailable, or because verification would take longer than the decision itself—a signature transfers accountability without improving the decision. Green [1] argues that human-oversight requirements often function as legitimacy theatre for this reason; the empirical literature on automation bias and appropriate reliance points the same way [2–4].

Our claim is that the failure is not of oversight but of *unit*. Reviewing a decision is work that must be repeated. Reviewing the *logic* that produces decisions is work that amortises.

2 Design

2.1 The logic database, and why it comes first

It is tempting to treat the inference engine as plumbing beneath the interesting governance question. That is backwards. Approving a rule is a promise about the future—this logic, on every case whose facts match it, from now on—and a promise of that shape is only redeemable if three things hold of the engine underneath.

The same inputs must give the same answer. Otherwise approval binds nothing. Evaluation is a terminating fixpoint over function-free Horn clauses: with no function symbols every derivable atom is built from constants already present, so the derivable set is finite and the fixpoint is reached in finitely many rounds. Termination is structural rather than a timeout, which matters because rules arrive from an agent over time—a recursive rule it proposes cannot hang the system. Each round matches against a snapshot of what was derived at the round’s start, so a rule’s result cannot depend on where in the round it happened to run, and insertion order cannot change the answer. Stratification is re-checked on *every* insertion rather than once at startup, for the same reason: an unstratifiable program has no unique minimal model, so “deterministic” would be a lie, and the rejection names the offending cycle so whoever proposed the rule can fix it.

A reviewer must be able to see what a rule did. Forward chaining is used rather than backward precisely so the whole derivation is available and not just an answer to the question asked. The explanation is a rendering of that derivation—if the text says a conclusion rests on three facts and a rule, it rests on exactly those, because the text is produced by walking the structure that produced the answer. Every leaf carries the source field it came from and every rule its approver and cited clause.

Missing evidence must not be silently converted into a decision. This is the part that took most of the engineering. Facts are three-valued, and the closed-world assumption is rejected: absence is recorded as UNKNOWN, not as false. The non-obvious requirement is that uncertainty must survive negation. Under stratified

negation with a closed world, a derived atom whose own derivation is blocked by a missing fact reads as *false*, so `not mitigated(A)` succeeds and an applicant is declined on a guarantor nobody ever recorded. We shipped this defect twice—once in the engine, once in the explanation layer that reported an assumed value as an established one. The engine now tracks blocked heads as a third state that propagates through negation, and the explanation marks an interval endpoint’s assumptions as assumptions.

2.2 Bounds, and the question worth asking

Where facts are unknown the system reports an interval $[L, U]$ spanning every way they could resolve, rather than a point estimate over an assumed completion. Computing that naively means evaluating all 2^k assignments of the k unknowns.

It can be done in two. For a negation-free program the probability of the goal is monotone in each literal, so the minimising assignment sets every unknown to `FALSE` and the maximising one to `TRUE`. With negation, monotonicity is decided per unknown by a polarity analysis over the rule dependency graph: an unknown reachable only positively is set directly, only negatively is set directly the other way, and only those reachable under *both* polarities need enumerating—in practice a handful. The analysis runs over the rule base rather than over one evaluation, because the derivation itself changes as unknowns are pinned. The fast path is checked against exhaustive enumeration on randomly generated programs including negation, since a shortcut nobody compares against the definition is a guess.

The interval width is then the value of the missing information, which converts “we cannot tell” into a specific request. For each unknown the answer is recomputed with it resolved both ways, and unknowns are ranked by whether resolving them moves the decision across the threshold. Unknowns that change nothing are not offered at all: a system that asks for information it will not use trains people to stop answering.

2.3 The architectural commitment

The system rests on one commitment: **the agent proposes; only the engine infers**. A language model may draft candidate facts and candidate rules. It never produces a decision. Inference is a terminating Datalog fixpoint over function-free Horn clauses with stratified negation, so identical facts and rules yield identical answers.

This is how determinism and explainability survive contact with a language model. Nondeterminism is quarantined upstream of a human approval gate; everything downstream is reproducible. A hallucinated predicate does not become a bad decision—the vocabulary is closed and validation runs before the review queue, so it becomes a discarded proposal that no reviewer ever sees.

The choice of Datalog over richer formalisms follows directly from Section 2.1. Prolog is Turing-complete, so a recursive rule an agent adds can fail to terminate—fatal once rules arrive from a model rather than a person. Answer Set Programming admits multiple answer sets, which undercuts the determinism approval depends on, and its explanations must be reconstructed after the fact. Datalog is decidable and terminating, and its proof tree is the explanation. The formalism was chosen for the properties the governance layer needs, not for familiarity.

The same reasoning rules out deciding cases with a SAT or SMT solver, which is otherwise the obvious mechanisation. A solver returns *a* satisfying model rather than *the* answer, so where several models exist the choice among them belongs to the solver’s search order rather than to the policy—and approval is a promise about what a rule will do, which a search-order-dependent answer cannot keep. A model is also not a derivation: the explanation would have to be reconstructed afterwards, reintroducing the post-hoc account the proof tree exists to avoid. Satisfiability is moreover NP-complete, whereas the fixpoint is polynomial and terminates structurally. Solvers are the right instrument here, but for checking the rule base rather than for deciding cases, and we return to that distinction in Section 5.

2.4 Rules carry their governance

A rule is not merely a clause. It carries provenance, an approver, a usage count, and an accuracy record:

```
rule r0003 [p=0.80, origin=mined, status=approved,
           by="reviewer-jd", support=214, used=5,
           right="3/4"]
high_risk(A) ← overdrawn(A), thin_savings(A).
```

Two fields carry most of the design’s weight. The usage count is the amortisation figure. The accuracy tally makes a rule’s strength auditable rather than asserted: a reviewer sees $p=0.80$ claimed against 3/4 observed and can judge the gap. Where the two diverge past a threshold, the rule returns for re-review—the single exception to permanence, justified because the evidence changed rather than because the reviewer is being asked to repeat themselves.

2.5 Identity, and why approve-once needs it

Rules are keyed by a *canonical* hash: variables renumbered by first appearance, body literals sorted. Under this key, `decline(A) ← overdrawn(A), thin_file(A)` and `decline(Z) ← thin_file(Z), overdrawn(Z)` are the same rule.

Without canonicalisation, “approve once” degrades silently. An agent re-proposes settled logic in different syntax, a reviewer approves it again, and the amortisation ratio collapses to one review per proposal while every component appears to work. Identity is the primary key in the persistence layer, so this is a constraint the database enforces rather than a convention the application remembers.

2.6 Criticality routes review; approval attaches to rules

Criticality is a property of the *query*; approval is a property of the *rule*. A rule proposed in a routine context becomes `PROVISIONAL` and is usable immediately, with offline review queued. A rule needed by a critical query becomes `PENDING_ONLINE` and *blocks* that query.

The resulting invariant: **a critical decision can only ever be produced by a derivation in which every rule is approved**. It is enforced by handing the engine a filtered rule set, so an unapproved rule cannot contribute even accidentally, and re-audited afterwards over the full query log.

The provisional lane is a deliberate risk: routine work proceeds on unreviewed logic. The cost is made enumerable rather than

hidden—rejecting a provisional rule lists every decision that used it, across sessions.

2.7 Missing information produces bounds, not guesses

Facts are three-valued: TRUE, FALSE, UNKNOWN. Absence is not falsity. Where facts are unknown, the system reports an interval $[L, U]$ spanning every way they could resolve, computed by pinning unknowns to their answer-minimising and answer-maximising values. For monotone programs this requires two evaluations rather than 2^k ; only unknowns appearing in both polarities are enumerated. The fast path is verified against exhaustive enumeration on randomly generated programs including negation.

The width of the interval is the value of the missing information, which makes the next action determinate rather than a matter of judgement:

```
Answer: P = [0.00, 0.70]
Missing information, ranked by whether it
would change anything:
app_gap has a guarantor [DECISIVE]
if true -> 0.00, if false -> 0.70
```

Unknowns that would change nothing are not offered. A system that asks for information it will not use trains people to stop answering.

2.8 Evidence follows provenance

Rules arrive from three origins, and each asks the reviewer a different question. A rule extracted from a written policy cites its clause: the question is *does this faithfully encode the clause?* A rule mined from labelled cases carries support, precision, examples and counterexamples: the question is *is this correlation also a reason?* The review interface renders them differently on purpose. Presented identically, a reviewer would answer the easier question in both cases.

2.9 One engine, two front ends

The system is operated through a control room: a single page showing the rule base, the cases still needing a person, an inspector that renders either a rule’s review evidence or a decision’s proof, and the accountability record. Screening runs in batches against the live engine, so the governance behaviour is visible rather than described—starting a campaign before the rules are approved produces blocked rows naming the rules each case is waiting on, and those rows clear when the rules are approved.

We adopted this layout deliberately from our own earlier client-side prototype, discussed in Section 3.5, because the operational framing was right even where what sat behind it was not. What changed is the referent: there a fleet of a hundred simulated agents screened CVs and every screened CV became a human review; here the fleet is the *rule base*, a rule is reviewed once, and the component that decides is the one that also writes the proof.

3 Evaluation

Three domain packs share the engine and no code: consumer credit over the UCI Statlog German Credit data [7] with a written lending

Table 1: Engine properties and what backs each. Counts are tests in the repository; the bounds row compares the fast path against exhaustive enumeration on randomly generated programs including negation.

Property	Tests
Bounds exact vs. brute-force enumeration	55
Termination and stratification	24
Determinism: same inputs, same answer	22
Proofs: provenance, assumed vs. established	7
Unknowns survive negation	5
Value of information	3
No circular self-support	1
Total	117

policy, a synthetic benefits-eligibility domain, and a recruitment campaign reproducing the scenario of our own earlier prototype.

3.1 The engine’s properties are checked, not asserted

Determinism and explainability are the claims everything else rests on, so they are tested as properties over generated programs rather than demonstrated on an example. Of 289 tests, 117 exist to establish them (Table 1).

Two are worth drawing out. The bounds check is the load-bearing one: the fast path replaces 2^k evaluations with two plus a small enumeration, and it is only sound if it agrees with the definition, so it is compared against exhaustive enumeration over random programs—with negation, because the polarity reasoning is exactly what could be wrong there. And a rule base cannot inflate its own confidence: while computing the probability of an atom, any path that arrives back at that atom contributes nothing, so a self-citing recursive rule adds no support rather than compounding.

3.2 Oversight amortises

Figure 1 shows the central result. Sixteen acts of review governed 589 decisions—36.8 decisions per review, with the ratio still improving as the rule base matured. The approach is not free at the outset: reviews are paid before any decision is made, so rule-level oversight is initially *more* expensive per decision and breaks even at 16. For a caseload of a few dozen it would not be worth the machinery; the argument is about volume.

We report this figure with a caveat that cost us an earlier draft. An initial version divided *rule applications* by reviews and obtained 66.7. Several rules fire per decision, so that ratio overstates the claim by exactly that factor. Both are now reported separately; the decision-denominated figure is the defensible one.

3.3 The miner recovers the right rules

The synthetic pack is generated from a rule set that is then withheld, which permits a question real data cannot answer: not *does this rule predict well*, but *is it the right rule*. Figure 2 shows all three generating rules recovered exactly and ranked first among eight proposals.

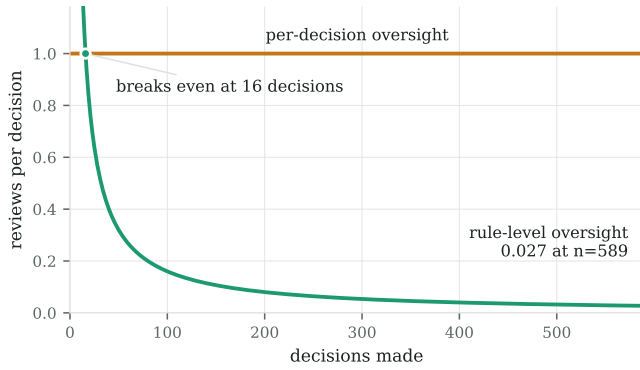


Figure 1: Oversight cost per decision over a run of 1,000 credit applications. Per-decision oversight costs one review per decision by definition. Rule-level oversight is measured: cumulative approved rules over cumulative decisions. Reviews are paid up front, so the approach is initially more expensive and breaks even after 16 decisions; at $n=589$ the cost is 0.027 reviews per decision, and still falling.

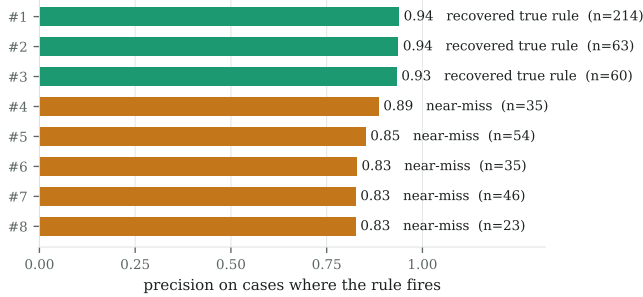


Figure 2: Mined proposals on the synthetic pack, ranked by precision. The three rules that generated the data are recovered exactly and rank first. Five plausible near-misses follow. Two attributes in the domain bear on nothing by construction; no proposal uses either.

The five near-misses are the interesting part, because two are wrong in ways a reviewer can check against written policy. One grants eligibility on a recent award, which policy clause s4.1 says should trigger *referral*. Another reproduces the disability route with the capital-limit guard silently dropped, and would pay applicants holding savings above the limit. Both sit at 83–85% precision and look entirely respectable. This is the concrete case for a human reading rules rather than trusting a threshold.

Reaching this result required a correction worth recording. An initial miner searched only positive conjunctions and therefore could not express `not savings_over_limit`. Rather than failing visibly, it *approximated* the rule with whatever correlated—including the two noise attributes—recovering one rule of three. Admitting negated literals took recovery to three of three and eliminated the noise proposals. A miner that cannot say “not” does not merely miss rules; it produces rules that are wrong in a way the evidence cannot reveal.

Table 2: The same 8,000-applicant campaign under both systems.

	Agent fleet	Rule base
One-off rule reviews	—	8
Human case reviews	5,858	2,558
Decided without a person	0	5,442
Screening logic inspectable	no	yes
Wrong rejections measurable	no	yes

3.4 The system found a gap in the written policy

We authored a lending policy, extracted it into eight cited rules, and observed that the system then declined to decide 11% of cases. Clause s7.1 covers elevated risk *without* mitigating security; s7.2 covers security *without* elevated risk; neither covers an applicant with both. We had not noticed.

This is the practical payoff of distinguishing “no applicable logic” from “no”. A scoring model would have produced a confident number for every one of those applicants. Competing outcomes are deliberately never normalised: if all are low, the rule base has nothing to say, and normalising near-zero values would disguise ignorance as a confident split.

3.5 The same campaign, against our own earlier prototype

Before building the rule base we had built a conventional agentic prototype for the same brief: a hundred simulated screening agents over an 8,000-applicant, 1,000-hire recruitment campaign, each agent able to reject a CV on a confirmed mismatch or flag it for a person. It is a useful baseline precisely because we wrote it, and because running it exposes two things its interface does not say.

First, it does not reduce human work. Of 8,000 CVs it flags 5,621 for review (70%), of which 4,012 are cases where the requirements were *met* and a person must still decide, plus 237 sampled rejection audits: 5,858 human reviews to make 1,000 hires, or 5.9 reviews per hire. Second, its interface states of agent rejections that “each can be reopened”, yet 2,142 of 2,379 rejections (90%) are never surfaced to any human, and the screening logic is a hard-coded function that cannot be inspected or approved at all.

We rebuilt the scenario as a third domain pack, with the screening policy written as prose and extracted into eight cited rules. Table 2 compares the two on the identical campaign.

The last row matters more than the counts. In the agent fleet a rejection is correct by construction: it fires only on an explicit unmet requirement, and there is no notion of a suitable applicant wrongly rejected, so an auditor has nothing to find and contestability can be offered but never exercised. The recruitment pack carries a withheld suitability label, and against it **26.4% of automatic rejections are of genuinely suitable applicants**. That is not a good number. It is, however, a number—and the audit queue now has something to find.

The pack also records two protected attributes that are deliberately *not* declared predicates, so no rule can be written or mined over either. Normalising by band size, applicants over 40 are auto-rejected *less* often than younger ones (28% against 38–39%), because

Table 3: Measured results. All figures are produced by scripts in the repository; the two figures are generated from live runs rather than transcribed.

Result	Value
Tests establishing engine properties	117 / 289
Bounds disagreeing with exhaustive enumeration	0
Decisions per act of review	36.8
Generating rules recovered exactly	3 / 3
Proposals resting on noise attributes	0 / 8
Cases on which written policy was silent	11%
Governance invariant violations	0

a timeline-conflict signal that correlates with age routes them to human review instead. A disparity that runs toward more human review is still a disparity, and it is visible only because the attribute is recorded while being unavailable to the logic.

3.6 Governance guarantees hold

Across every critical decision in the credit run, the invariant audit reports zero violations. The property is also checked directly: the test suite constructs a state in which an approved rule is retroactively downgraded and confirms the audit detects it, so the check is capable of failing.

4 Design principles

Each of the following was earned by a defect we shipped and then fixed, not derived from first principles.

- (1) **Put oversight where it amortises.** If oversight cost per decision is flat, the unit is wrong.
- (2) **Absence is not denial.** “No applicable logic” and “no” are different answers and must read differently.
- (3) **Unknowns must survive negation.** A fact blocked by a missing input is itself unknown. We shipped this wrong twice: a negated condition succeeded because nobody had recorded the fact underneath it, and an applicant was declined on an assumption no one had made.
- (4) **Approve the logic, not the text.** Identity must be invariant under renaming and reordering, or approve-once means nothing.
- (5) **Evidence follows provenance.** Different origins pose different questions and need different evidence.
- (6) **Make guarantees structural, not procedural.** Our lending policy forbids decisions resting on sex. That clause is deliberately *not* a rule—a rule can state a condition but not the absence of a hidden correlation. Instead the attribute is absent from the vocabulary, so no rule can be written or mined over it. Review cannot reliably detect a proxy and should not be asked to.
- (7) **State the assumption in the output.** Combining independent supports by noisy-OR is a modelling commitment and frequently a wrong one; it appears in every explanation rather than inside the arithmetic.

- (8) **A system that cannot be wrong cannot be audited.** Screening logic whose rejections are correct by construction offers contestability as an affordance and can never exercise it. Build the ground truth that lets a decision be wrong, or the review queue is decoration.
- (9) **The price of speed must be enumerable.** Provisional rules keep routine work moving; rejecting one names every decision it touched.

5 Limitations

No human has reviewed a rule in this system. The amortisation figure is arithmetic over a working prototype, not evidence that reviewers review rules well. If they cannot, the argument fails, and nothing in the present work would detect it.

The experiment that would settle it is unusually clean, because the synthetic pack supplies ground truth for *rules*: reviewers are shown the eight proposals of Figure 2, and the measurement is whether they approve the correct three and reject the five near-misses, at what time cost, and whether showing support and precision changes their accuracy. We regard this as the necessary next step rather than future work.

The rule base is validated case-by-case, not exhaustively. Safety, stratification and vocabulary are checked on every insertion, but three properties of the base *as a whole* are not: whether any input derives two competing outcomes at once, whether a proposed rule is already subsumed by an approved one, and whether an approved rule can ever fire. Each protects reviewer attention, which is the resource the whole design is trying to spend well, and each is an existential question over all inputs—the shape a solver is built for.

At present they can be answered without one. Every predicate in the three packs has arity one, so grounding on a single entity leaves a propositional problem over ten to nineteen observables, and one engine evaluation costs 0.137 ms: exhausting the recruitment pack’s 2^{10} inputs takes 0.1 s, and the credit pack’s 2^{19} takes 72 s. Running the engine itself over every input has a property a solver cannot match—enumeration cannot disagree with production behaviour, because it *is* production behaviour, and there is no encoding to get wrong. This is the same device that validates the probability bounds.

Enumeration dies exponentially all the same, and a pack with more than roughly twenty-five observables, or any predicate of arity two, will need a solver. The encoding is the part to be careful with: a solver reads $h \leftarrow b$ as material implication and admits models in which h holds for no reason, while the stratified semantics takes the unique minimal model. A naive translation reports contradictions the engine would never produce and misses real ones, so Clark completion with loop formulas is required—and the enumerator should be retained as the oracle the solver is validated against, exactly as it is for bounds.

Credit assignment is crude. Outcome feedback marks every rule in a derivation right or wrong together, so a determination rule is blamed for failures caused by its premises. Proper attribution requires something like a Shapley value over the derivation.

Rejection flags but does not reverse. The system enumerates the decisions a withdrawn rule touched and stops. Whether that invalidates them is a judgement it should not make alone.

The recruitment comparison is not like-for-like in every respect. The two systems partition work differently, and the baseline is our own prototype rather than a deployed product. What the comparison establishes is narrower than a benchmark and still worth stating: on identical inputs, moving review from cases to rules cut human case reviews by 56% while making the screening logic inspectable and its error rate measurable for the first time.

Accuracy figures on the credit pack are not a benchmark. They score an invented lending policy against real outcome data. The system detecting that its own policy is poorly fitted is the machinery working, not a capability claim.

6 Related work and positioning

Datalog evaluation is well understood [5] and we borrow it unchanged, which is precisely why the determinism claim holds. Each of the engine's other properties also has antecedents: a third truth value is familiar from well-founded and partial semantics, probabilistic logic programming has a substantial literature, and value of information is standard in decision analysis.

What we claim for the logic database is not any one of these in isolation but their composition under a specific constraint—that the output be reviewable by a person who did not write it. That constraint is what forces uncertainty to survive negation rather than collapsing to false at the first negated literal, what makes exact bounds worth computing cheaply rather than approximating, what turns the interval width into a single named question instead of an error bar, and what makes the proof distinguish an assumption from a finding. Those choices do not follow from wanting a fast solver. They follow from wanting an answer a reviewer can check.

The governance layer is the second contribution: approve-once identity by canonical hashing, criticality-routed review, enforced approval before any critical use, and retroactive impact analysis. It is separable in presentation and not in practice—it is only sound on top of an engine with the properties above.

Classical expert systems foundered partly on knowledge acquisition [6]. A language model that drafts candidate rules under structured review is a different answer to that bottleneck than a knowledge engineer with a notepad—and, critically, one whose output is governed rather than trusted. Relative to work on appropriate reliance [3, 4], which studies how humans respond to per-decision advice, we change the object of review rather than its presentation.

7 Conclusion

Moving human oversight from decisions to the logic that produces them makes oversight amortise, makes decisions explainable by construction rather than by narration, and makes it possible to guarantee structurally that a consequential decision never rests on logic nobody approved. The prototype demonstrates that these properties are jointly achievable.

The two contributions are not parallel. A reviewer can approve a rule once because the engine guarantees that approval binds the rule's behaviour on every future case, and can approve it *responsibly* because the engine will show them what it did and will not

quietly convert a missing fact into a decision. Determinism and explainability are usually argued for as desirable properties of a decision system. Here they are load-bearing: without them, rule-level oversight is not merely less pleasant, it is unavailable. Whether humans review rules well enough to realise the benefit is the open question, and it is answerable.

References

- [1] Ben Green. 2022. The Flaws of Policies Requiring Human Oversight of Government Algorithms. *Computer Law & Security Review* 45.
- [2] Raja Parasuraman and Victor Riley. 1997. Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors* 39, 2.
- [3] Zana Bućinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. 2021. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. *Proc. ACM HCI* 5.
- [4] Gagan Bansal et al. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. *CHI*.
- [5] Stefano Ceri, Georg Gottlob, and Letizia Tanca. 1989. What You Always Wanted to Know About Datalog. *IEEE TKDE* 1, 1.
- [6] Edward A. Feigenbaum. 1980. Knowledge Engineering: The Applied Side of Artificial Intelligence. Stanford University.
- [7] Hans Hofmann. 1994. Statlog (German Credit Data). UCI Machine Learning Repository.